TP GitHub Projects CI-CD

Nom et Prénom : Maxence Grimal
Intitulé du TP : Découvrir l'intégration continue avec GitHub Actions
Projet : TP_GitHub-Projects_CI-CD

Nom de dépôt GitHub : "TP_GitHub-Projects_CI-CD"

URL du dépôt : "https://github.com/maxencegrimal/TP_GitHub-Projects_CI-CD"

Branche principale : main

1. Réponses aux Questions du TP

Question 1 : Différence entre git init, git add et git commit

- git init : Initialise un nouveau dépôt Git vide dans le répertoire courant.
- git add : Place les fichiers créés ou modifiés dans la zone de transit (staging area / index) pour préparer le commit.
- git commit : Enregistre définitivement l'état des fichiers indexés dans l'historique local du projet.

Question 2 : Notions de dépôt local et distant

1. Dépôt local vs distant : Le dépôt local réside sur le disque dur de la machine de travail. Le dépôt distant est hébergé en ligne sur GitHub pour sauvegarder, partager et centraliser le code.
2. Rôle de origin : Nom raccourci (alias) attribué par défaut à l'URL du dépôt distant distant sur GitHub.
3. Rôle de git push : Envoie les commits de la branche locale vers le dépôt distant.
4. Option -u : Associe la branche locale à la branche distante (upstream), permettant d'exécuter ultérieurement un simple git push ou git pull sans répéter la destination.

Question 3 : Environnement virtuel et gestion des exceptions

- Intérêt de l'environnement virtuel (.venv) : Isoler les dépendances Python du projet (comme pytest) pour éviter tout conflit avec l'environnement système global de la machine.
- Test de division par zéro : Le test réussit car il utilise pytest.raises(ValueError), une assertion qui valide que l'exception attendue est bel et bien levée par le programme lors d'une division par zéro.

Question 4 : Déclenchement automatique de la CI

La CI se déclenche dès le premier git push sur main car le fichier .github/workflows/ci-cd.yml contient l'instruction on: push: branches: [main]. GitHub Actions intercepte l'événement d'envoi de code et exécute automatiquement la série de vérifications paramétrées.

Question 5 : Diagnostic de la régression et logs

- Détection de la régression : Le test unitaire test_add attendait la valeur 5 pour add(2, 3). La fonction ayant été modifiée avec une soustraction, elle a retourné -1, provoquant l'échec d'assertion (assert -1 == 5).
- Apport des logs : L'icône rouge signale un échec global, tandis que le journal d'exécution (logs) fournit le nom exact du test défaillant, la ligne de code en cause, ainsi que les valeurs attendue et obtenue.

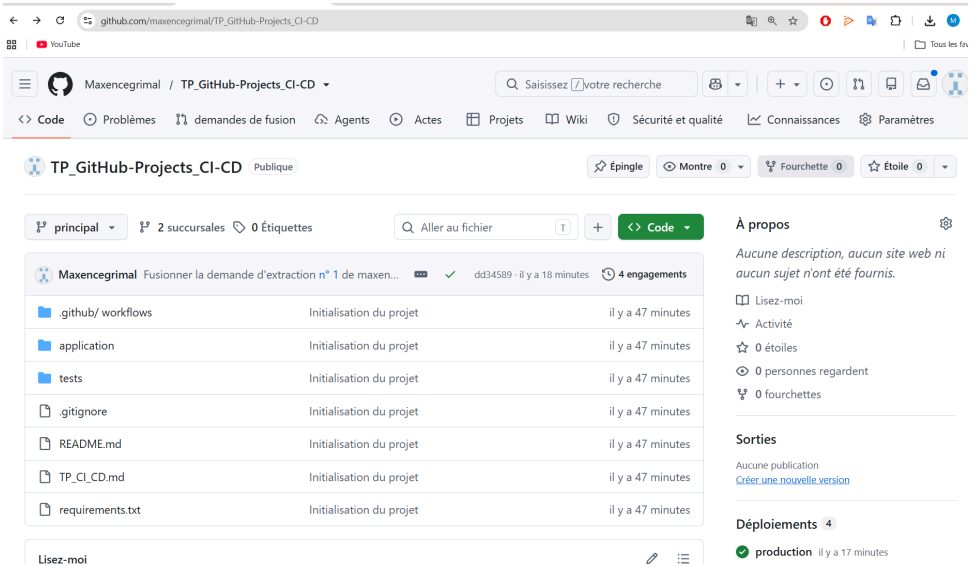
2. Diagnostic et Correction du Bug (Étapes 6 & 7)

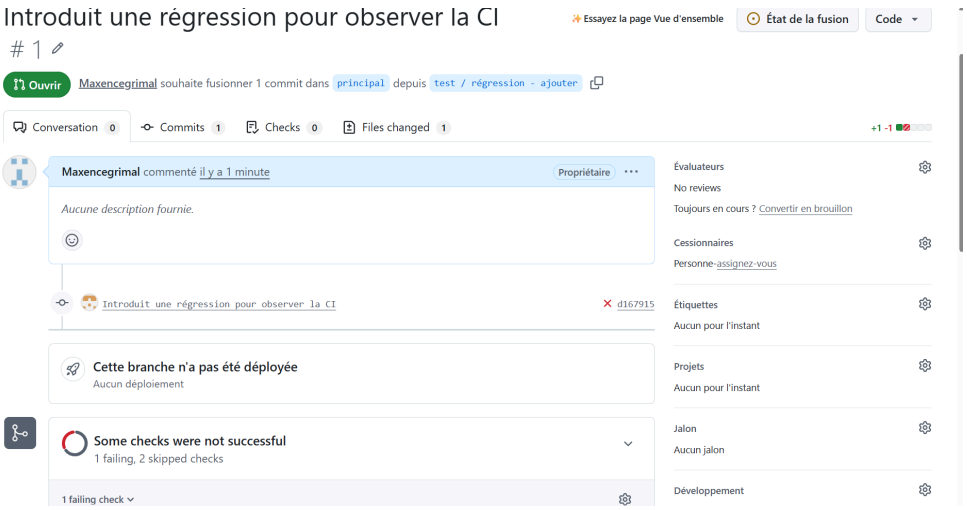
Test en échec : test_add dans tests/test_calculator.py.

Cause du problème : Dans app/calculator.py, l'opérateur d'addition a été temporairement remplacé par une soustraction (return a - b).

Correction apportée : Restitution de l'addition (return a + b) dans app/calculator.py, documentation de l'incident dans README.md, puis commit et push vers la branche de Pull Request.

3. Preuves d'Exécution et Captures d'Écran







4. Conclusion

Un test local s'exécute manuellement sur la machine de développement (via python -m pytest). À l'inverse, l'Intégration Continue (CI) exécutée par GitHub Actions garantit la qualité du code de façon automatisée et neutre sur un serveur distant vierge à chaque modification, interdisant la fusion de tout code défaillant dans la branche principale.